

SEMANA 3 · TUTORÍA SINCRÓNICA

# Modelos de Aprendizaje No Supervisado

Sábado 03/10/2026 · 3 h      MIAR0525 · Maestría en Inteligencia Artificial

# Agenda de hoy

| Tiempo    | Bloque                                                   | Animaciones |
|-----------|----------------------------------------------------------|-------------|
| 0:00–0:10 | Repaso de la semana 2; ¿qué es "correcto" sin etiquetas? | —           |
| 0:10–0:50 | K-means y escalamiento                                   | A3.1 · A3.2 |
| 0:50–1:20 | DBSCAN y validación de clusters                          | A3.3 · A3.4 |
| 1:20–1:30 | Pausa                                                    | —           |
| 1:30–2:10 | PCA y t-SNE                                              | A3.5 · A3.6 |
| 2:10–2:40 | Anomalías: reglas, Isolation Forest y LOF                | A3.7 · A3.8 |
| 2:40–3:00 | Ética de segmentar y marcar personas; Tarea 3            | —           |

# Tres errores que conviene no repetir

- 1 **Elegir hiperparámetros o el umbral con la prueba.** La búsqueda y el umbral se deciden con validación cruzada sobre el entrenamiento; la prueba se usa una vez.
- 2 **Declarar un ganador por milésimas.** Si dos modelos quedan dentro de una desviación estándar, se prefiere el más simple o explicable.
- 3 **Decidir con probabilidades sin calibrar.** La regla  $p > c/g$  solo funciona si  $p$  es una probabilidad creíble (Brier, diagrama de fiabilidad).

# Sin $y$ , ¿cómo sabemos si está bien?

## Interna

Geometría con los mismos datos: inercia, silhouette, Davies-Bouldin, varianza explicada.

## Estabilidad

¿Sale lo mismo con otra semilla u otra muestra?  
ARI entre bootstraps.

## Externa y utilidad

Información que no entró al algoritmo y la pregunta: ¿sirve para decidir algo?

## El algoritmo siempre encuentra algo

K-means devuelve  $k$  grupos aunque los datos sean ruido. Una solución no supervisada es una **hipótesis** que hay que validar.

# Tres tareas y una responsabilidad

## Clustering

K-means y DBSCAN:  
segmentos de clientes,  
zonas geográficas.

## Reducción

PCA para resumir y  
comprimir; t-SNE solo  
para mirar.

## Anomalías

Isolation Forest y LOF:  
fraude, intrusiones,  
fallas.

## La responsabilidad (RDA2)

Segmentar o marcar como "raras" a personas tiene consecuencias. Auditamos la composición por subgrupo antes de usar cualquier resultado.

BLOQUE 1 · 0:10–0:50

# K-means y escalamiento

Centros, distancias y la decisión de cuánto pesa cada variable.

# k centros que minimizan la inercia

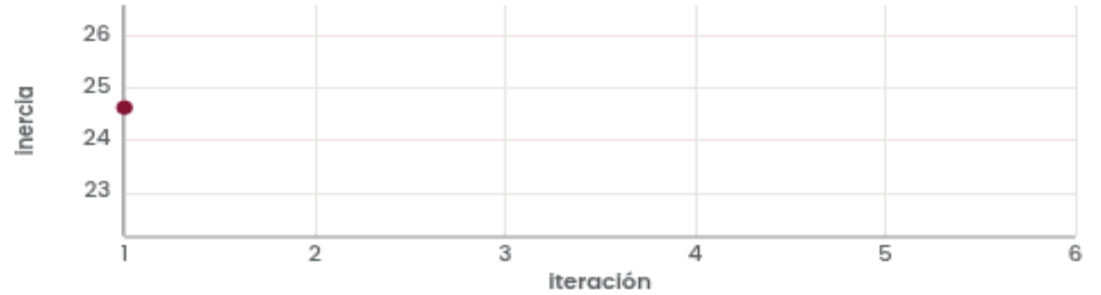
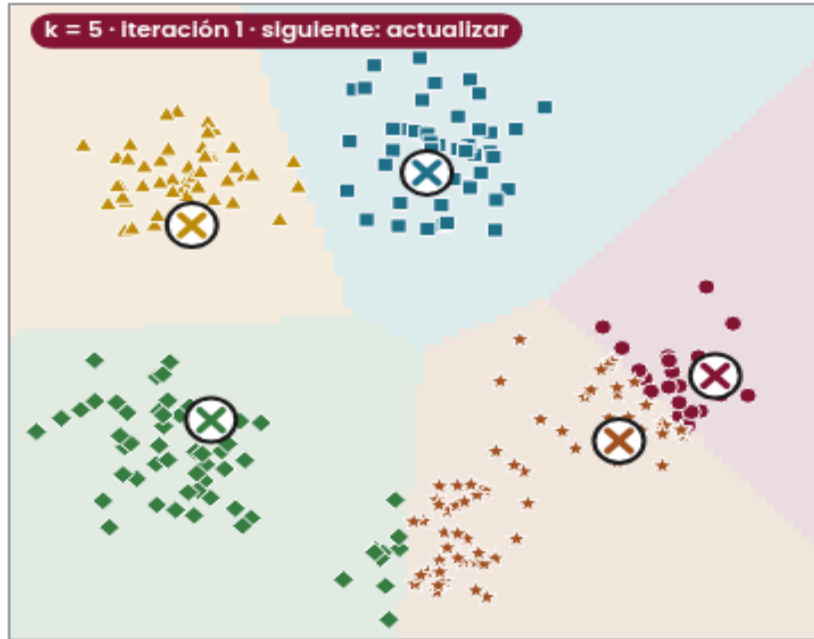
$$J = \sum_{j=1}^k \sum_{x \in C_j} \|x - \mu_j\|^2$$

- 1 **Asignar:** cada punto al centroide más cercano.
- 2 **Actualizar:** cada centroide a la media de sus puntos.
- 3 Repetir hasta que nada cambie.  $J$  nunca sube.

## Pregunta

Si  $J$  nunca sube y el algoritmo siempre converge, ¿por qué dos corridas pueden dar resultados distintos?

# Asignar y actualizar



Aquí aparecen la comparación de inicios (escena 3) y el codo con la silhouette (escena 4).

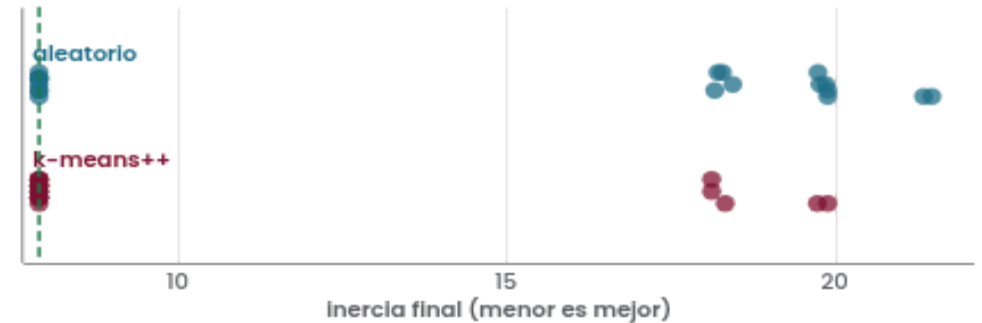
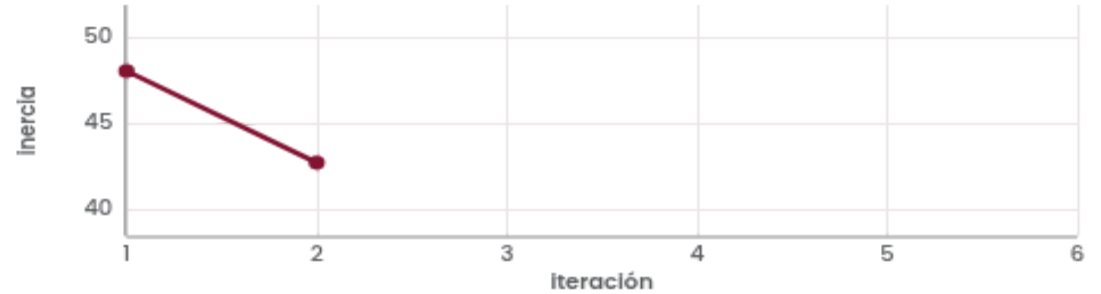
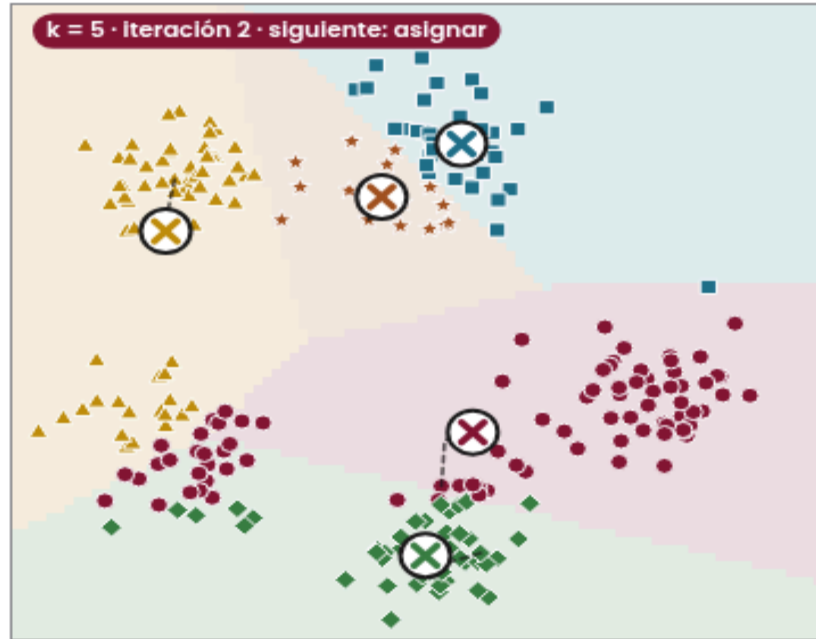
# Óptimos locales y k-means++

- **k-means++**: el primer centro al azar; cada nuevo centro con probabilidad  $\propto D(x)^2$  (lejos de los ya elegidos).
- Varias inicializaciones ( `n_init` ) y se queda la de menor inercia.
- Con `n_init="auto"`, scikit-learn hace una sola corrida con k-means++: conviene fijar `n_init=10`.

## E3.1 · 5 grupos, 30 corridas

| Inicio                                | Llega al mejor<br>óptimo | Peor<br>inercia |
|---------------------------------------|--------------------------|-----------------|
| Aleatorio                             | 63 %                     | 2295.7          |
| k-means++                             | <b>90 %</b>              | 1759.2          |
| Mejor inercia en ambos casos: 1035.6. |                          |                 |

# Veinte inicios aleatorios frente a veinte k-means++

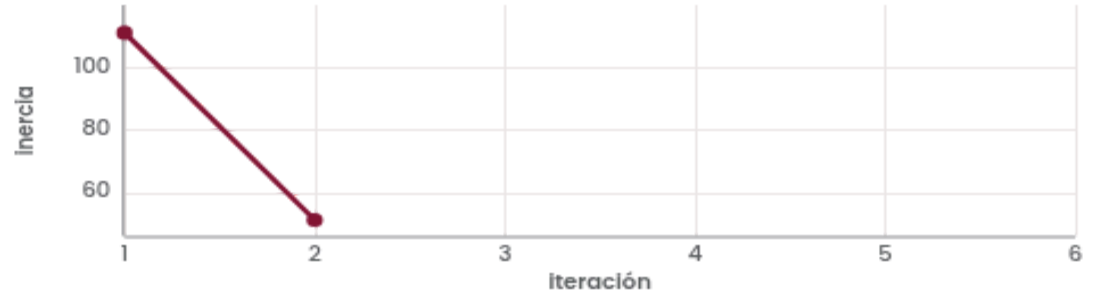
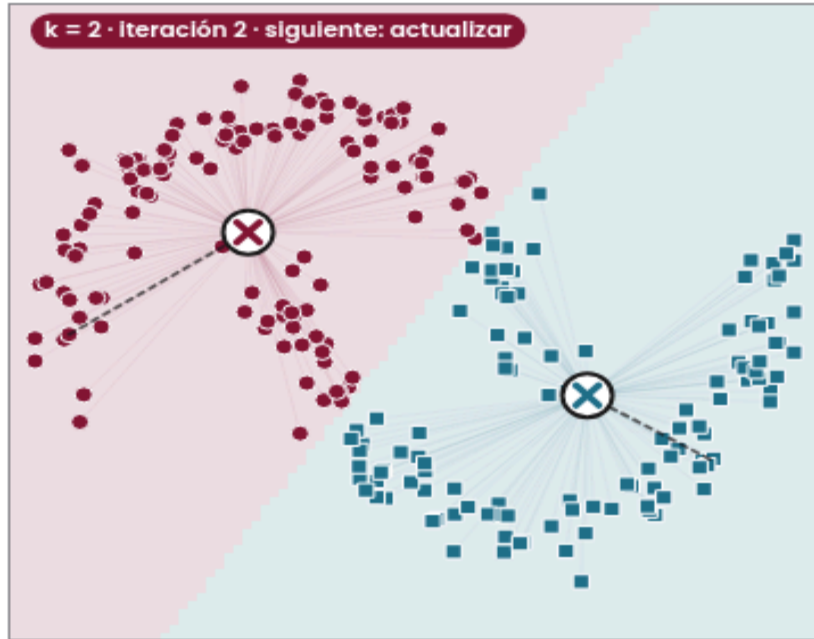


# Codo, silhouette, Davies-Bouldin y estabilidad

| k        | Inercia | Silhouette<br>↑ | Davies-Bouldin<br>↓ | ARI<br>bootstraps |
|----------|---------|-----------------|---------------------|-------------------|
| 3        | 3204    | 0.546           | 0.610               | 0.998             |
| 4        | 1856    | 0.590           | 0.537               | 0.993             |
| <b>5</b> | 1036    | <b>0.612</b>    | <b>0.516</b>        | 0.988             |
| 6        | 935     | 0.553           | 0.749               | 0.898             |
| 8        | 768     | 0.429           | 0.991               | 0.769             |

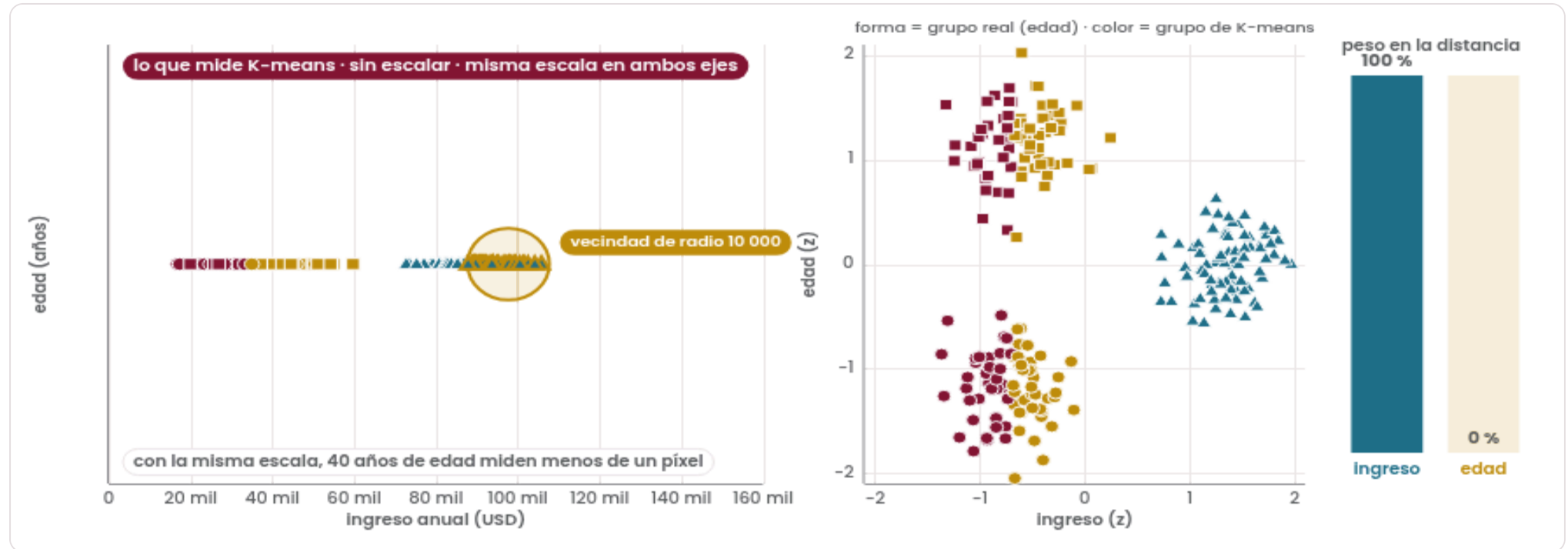
- La inercia **siempre** baja con k: el codo es una pista, no una prueba.
- Con el k verdadero (5), silhouette máxima, Davies-Bouldin mínimo y solución estable.
- Con k equivocado, cada muestra parte los grupos de otra forma: el ARI cae.
- En datos reales los índices rara vez coinciden: decide también la **utilidad**.

# Grupos convexos, parecidos y sin ruido



Aquí aparecen la comparación de inicios (escena 3) y el codo con la silhouette (escena 4).

# Ingreso en dólares, edad en años



# La escala es la decisión de cuánto pesa cada variable

- **StandardScaler:** media 0 y desviación 1.  
Sensible a atípicos.
- **RobustScaler:** mediana e IQR; los extremos no deforman al resto.
- **Log o QuantileTransformer** para montos con cola larga: en E3.1 el saldo va de -8 019 a 102 127 con mediana 448.
- **Categorías:** one-hot con cuidado, o úsalas para describir los grupos en lugar de formarlos.
- Nunca identificadores ni códigos numéricos como si fueran magnitudes.

Python

```
from sklearn.cluster import KMeans
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import QuantileTransformer

seg = make_pipeline(
    QuantileTransformer(n_quantiles=200, random_state=2026)
    KMeans(n_clusters=4, n_init=10, random_state=2026))
etiquetas = seg.fit_predict(X_comportamiento)
```

BLOQUE 2 · 0:50–1:20

# DBSCAN y validación de clusters

Clusters como regiones densas, y cómo saber si una partición es buena.

# Núcleo, borde y ruido

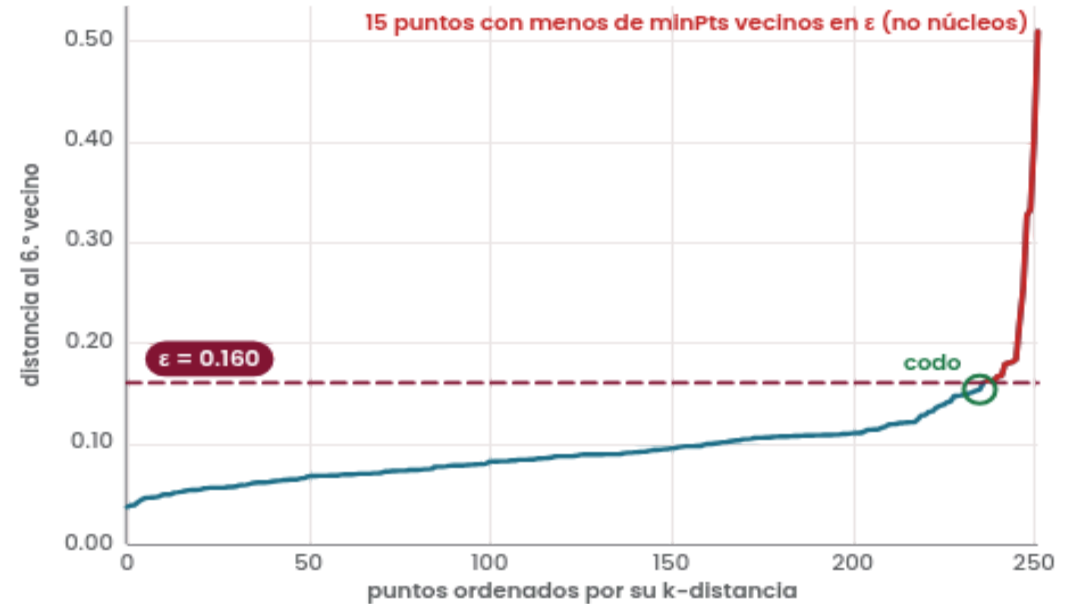
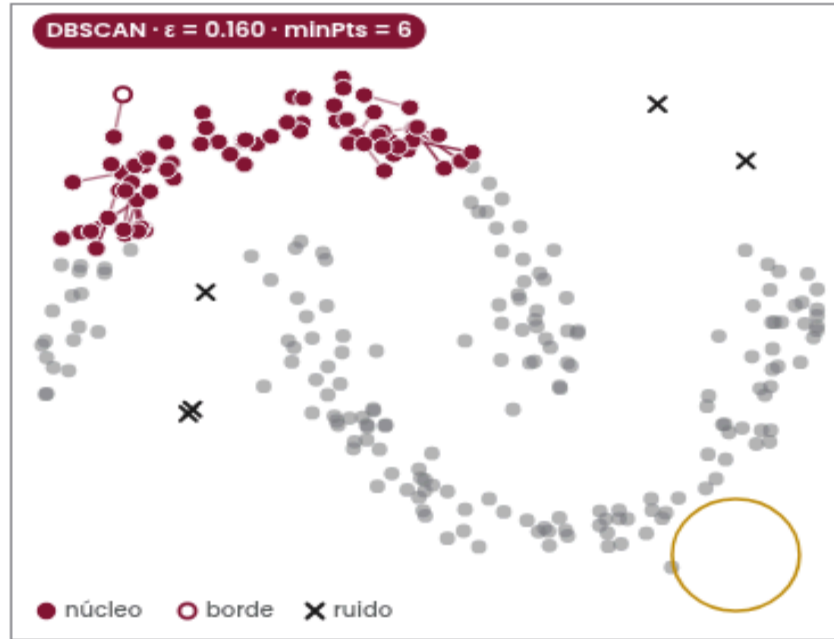
$$N_{\varepsilon}(p) = \{ q : d(p, q) \leq \varepsilon \}$$

- **Núcleo:**  $|N_{\varepsilon}(p)| \geq \text{minPts}$  (contando al propio punto).
- **Borde:** no es núcleo, pero está en la vecindad de un núcleo.
- **Ruido:** todo lo demás (etiqueta -1).
- Un cluster crece en anchura desde un núcleo; los bordes se agregan pero no expanden.

## Ventajas y precio

No pide  $k$ , encuentra formas arbitrarias y separa el ruido. A cambio, depende mucho de  $\varepsilon$  y sufre con densidades distintas.

# Un cluster crece de núcleo en núcleo



## El codo de la k-distancia y la distancia haversine

- **k-distancia:** distancia de cada punto a su k-ésimo vecino ( $k = \text{minPts}$ ), ordenada. El codo separa lo denso del ruido. En E3.2: percentil 90 = 0.145, 95 = 0.186  $\rightarrow \epsilon = 0.15$ .
- **minPts:** al menos la dimensión + 1; el doble con datos ruidosos.
- **Coordenadas:** `metric="haversine"`, `[lat, lon]` en radianes y  $\epsilon = \text{km} / 6371$ .

Python

```
from sklearn.cluster import DBSCAN

coords = np.radians(df[["Latitude", "Longitude"]])
db = DBSCAN(eps=5 / 6371, min_samples=60,
            metric="haversine",
            algorithm="ball_tree").fit(coords)
# California: 22 zonas densas, 29.7 % de ruido
```

## No hay un $\epsilon$ correcto en abstracto

| $\epsilon$ (km) | minPts | Zonas | % ruido |
|-----------------|--------|-------|---------|
| 3               | 120    | 5     | 87.2    |
| 5               | 60     | 22    | 29.7    |
| 10              | 60     | 21    | 16.3    |
| 20              | 30     | 8     | 3.2     |

Con  $\epsilon = 20$  km, una sola zona (de San Diego a Santa Bárbara) reúne el 53 % de los distritos.

¿Barrios, ciudades o regiones? **La pregunta fija  $\epsilon$ .**

Estabilidad en submuestras del 80 %: ARI 0.96 (5 km), 0.98 (10 km), 0.94 (20 km).

# Silhouette: ¿más cerca de los tuyos que de los vecinos?

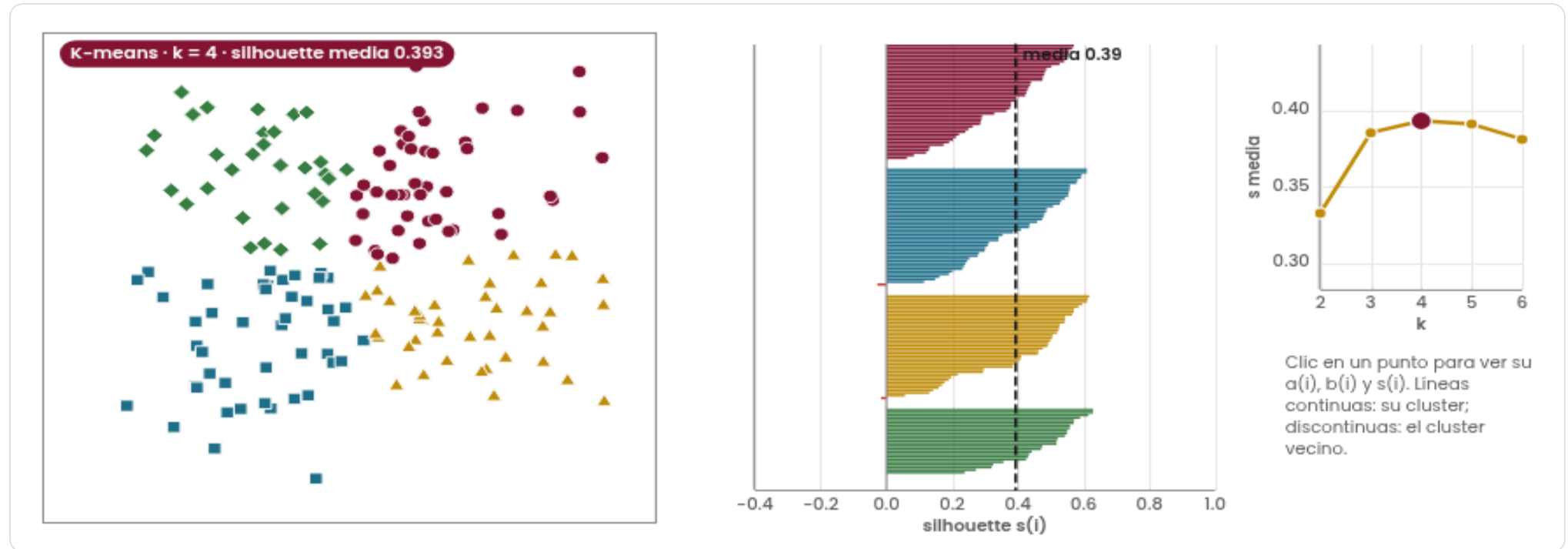
$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1]$$

- $a(i)$ : distancia media a su propio cluster (cohesión).
- $b(i)$ : distancia media al cluster vecino más cercano (separación).
- Cerca de 1: bien ubicado · cerca de 0: en la frontera · **negativa: mal asignado.**

## En la práctica

Costo cuadrático en n: con 45 000 clientes,  
`silhouette_score(..., sample_size=8000)`.

# s(i) punto por punto y el gráfico de silhouette



## Cuatro segmentos que se describen con una frase

| Segmento                           | % clientes | Saldo mediano | Contratación  |
|------------------------------------|------------|---------------|---------------|
| Con hipoteca, nunca contactados    | 36.4       | 420           | 6.7 %         |
| Sin hipoteca ni préstamo           | 31.8       | 522           | 13.4 %        |
| Contactados en campañas anteriores | 18.3       | 602           | <b>23.1 %</b> |
| Con préstamo personal              | 13.5       | 238           | 5.9 %         |

Silhouette sube hasta  $k = 8$ ; Davies-Bouldin mejora poco a poco: **los índices no eligen.**

$k = 4$ : estable (ARI 0.88) y útil. La contratación no entró al modelo: **validación externa.**

1:20–1:30

# **Pausa · 10 minutos**

Al volver: ¿cómo mirar 64 variables a la vez?

BLOQUE 3 · 1:30–2:10

# PCA y t-SNE

Resumir con PCA; mirar con t-SNE, sin creerle de más.

# Las direcciones de máxima varianza

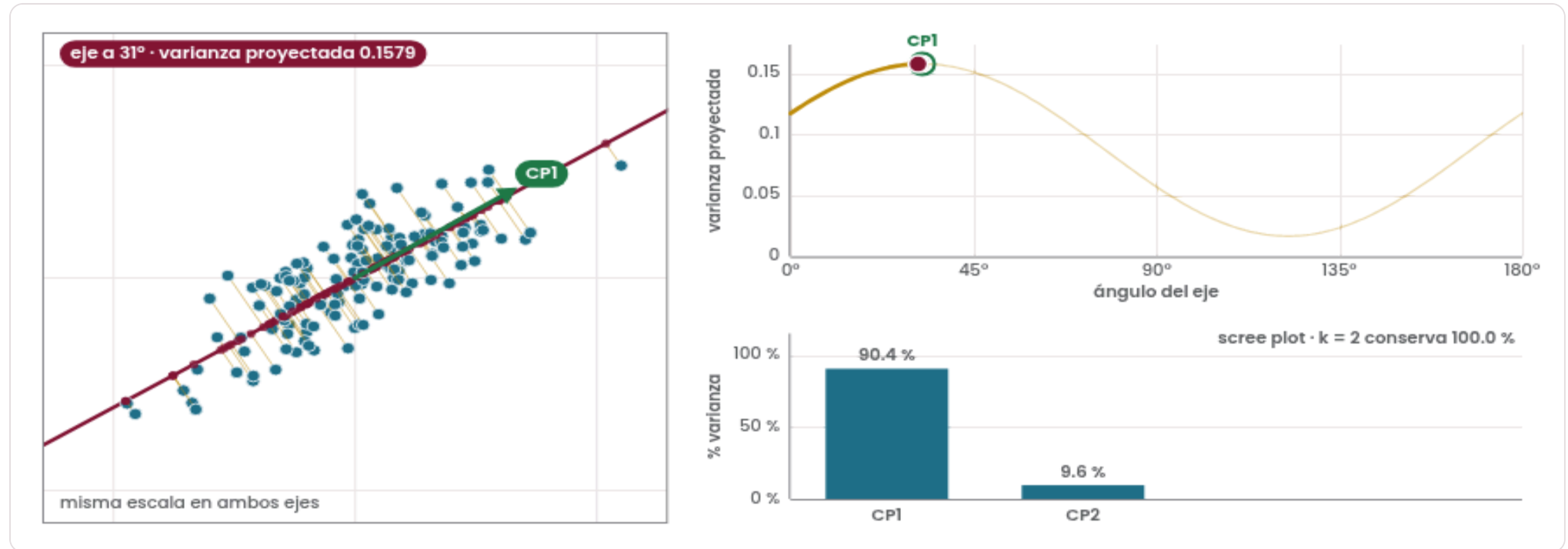
$$X_c = U\Sigma V^\top \quad Z = X_c V_k \quad \hat{X} = ZV_k^\top + \bar{x}$$

- Columnas de  $V$ : las **componentes** (vectores propios de la covarianza).
- Varianza de la componente  $j$ :  $\sigma_j^2 / (n - 1)$  ; se reporta como proporción.
- Máxima varianza proyectada = **mínimo error de reconstrucción**.

## E3.3 · nube 3D

Varianza: 85.6 % · 13.9 % · 0.6 %. Error de reconstrucción: 1.565 con  $k = 1$ , 0.061 con  $k = 2$ .

# Un eje que gira hasta capturar la mayor varianza



## Varianza acumulada, codo y el modelo que viene después

- **Olivetti** (4 096 píxeles): 10 / 50 / 150 componentes explican 65.6 / 87.4 / 96.3 %.  
Con 50 la cara ya se reconoce.
- **digits** (64 píxeles): 2 componentes, 28.5 %; hacen falta 21 para el 90 %.
- `PCA(n_components=0.95)` elige k por varianza.
- Estandariza antes y ajusta PCA **dentro del pipeline**.

| Componentes | Exactitud CV (logística) |
|-------------|--------------------------|
| 5           | 0.815 ± 0.019            |
| 10          | 0.879 ± 0.018            |
| 20          | 0.943 ± 0.012            |
| 30          | 0.955 ± 0.010            |
| 64 (todas)  | 0.971 ± 0.006            |

# Que los vecinos sigan siendo vecinos

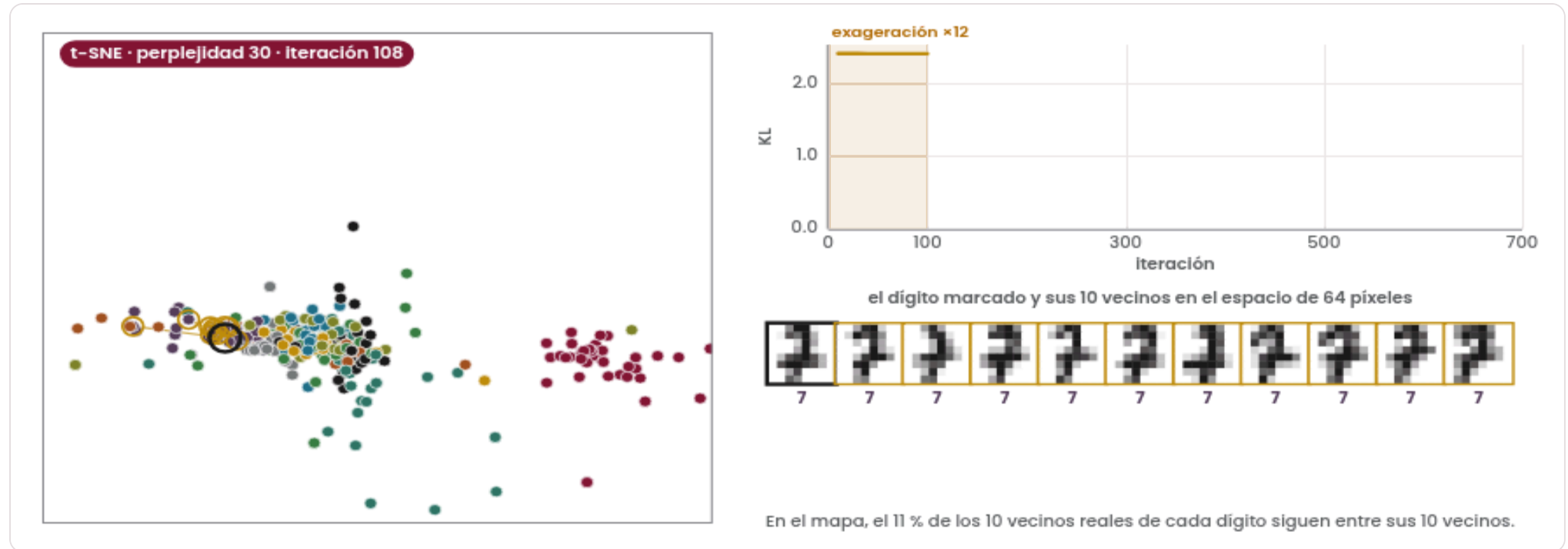
$$p_{j|i} \propto e^{-\|x_i - x_j\|^2 / 2\sigma_i^2} \quad q_{ij} \propto (1 + \|y_i - y_j\|^2)^{-1} \quad \min \text{KL}(P \parallel Q)$$

- **Perplejidad** ( $\approx$  número efectivo de vecinos) fija cada  $\sigma_i$ . Valores típicos 5–50; por defecto 30.
- En 2D, una t de Student de colas pesadas: los grupos se separan.
- La KL castiga separar vecinos, no juntar lejanos: conserva lo **local**.

## Pregunta

Si la KL casi no castiga poner cerca a dos puntos lejanos, ¿qué podemos leer de la distancia entre dos islas?

# 360 dígitos manuscritos se ordenan en islas

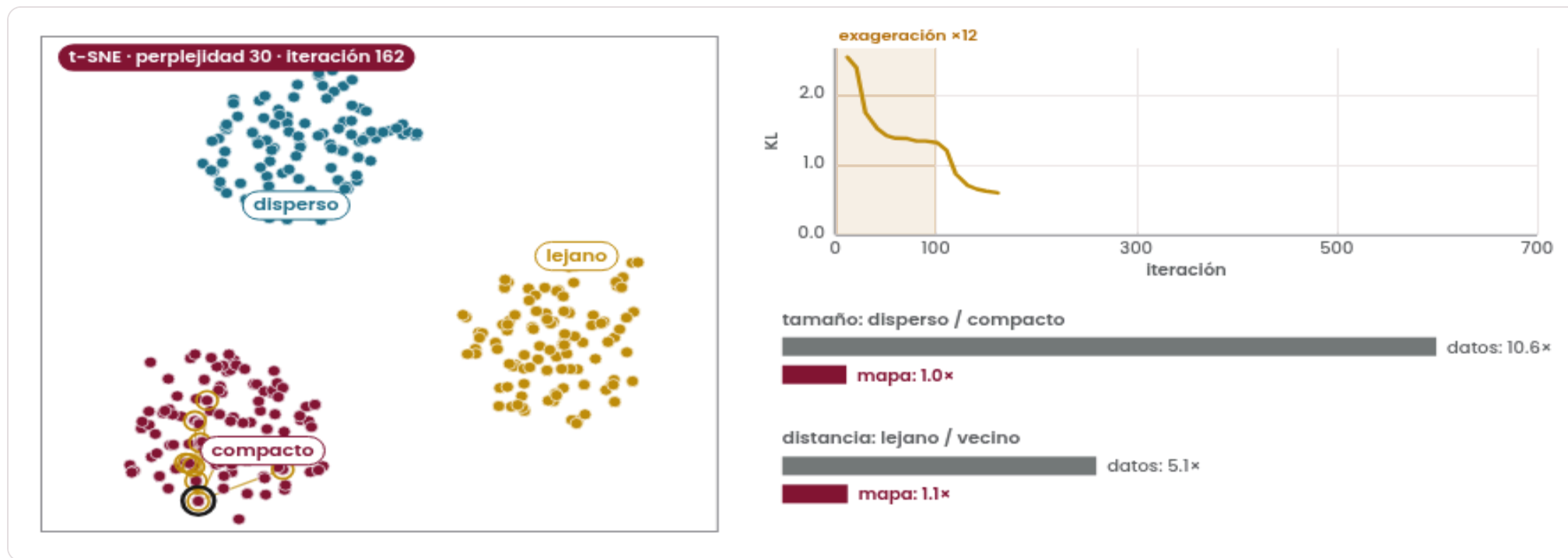


# Tamaños y distancias que no significan nada

| Espacio               | Disperso / compacto (tamaño) | Lejano / vecino (distancia) |
|-----------------------|------------------------------|-----------------------------|
| Datos originales (5D) | 9.5                          | 17.2                        |
| PCA 2D                | 9.8                          | 17.2                        |
| <b>t-SNE 2D</b>       | <b>1.4</b>                   | <b>2.8</b>                  |

En un mapa de t-SNE se lee **quién está cerca de quién**. No se leen tamaños, distancias entre grupos, posición ni densidad.

# Compacto, disperso y lejano: el mapa los iguala



# Para explorar, nunca como variables

- 1 Al menos **dos perplejidades** (p. ej. 5 y 50) y más de una semilla; `init="pca"`.
- 2 Escala antes; con muchas variables, reduce primero con PCA a  $\approx 50$  componentes.
- 3 TSNE no tiene `transform`: sus coordenadas no entran a un modelo.
- 4 Toda afirmación ("el segmento A es muy distinto del B") se comprueba en el espacio original.

BLOQUE 4 · 2:10–2:40

# Detección de anomalías

Lo raro importa, y cada alerta tiene un costo.

## Empezar por lo simple

- **Atípicos:** el entrenamiento ya contiene anomalías. **Novedades:** se entrena con datos verificados como normales.
- **z-score > 3:** supone normalidad; los atípicos inflan la media y la desviación (enmascaramiento).
- **IQR:** robusto, pero marca la cola de las variables asimétricas.
- Ninguna regla univariada ve anomalías de **combinación**.

| E3.4 · 50 anomalías | Detecta | Falsas alarmas |
|---------------------|---------|----------------|
| z-score > 3         | 33      | 0              |
| IQR 1.5             | 50      | 25             |
| IQR sobre log x     | 45      | 4              |

# Lo raro se aísla con pocos cortes

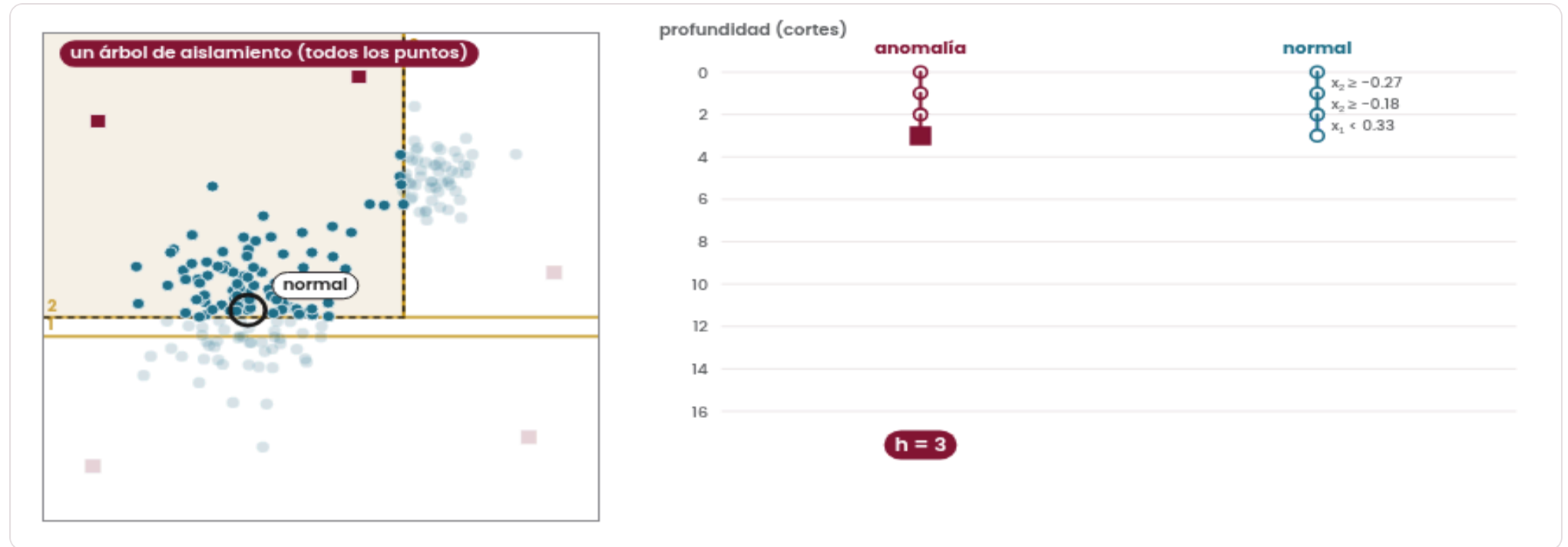
$$s(x) = 2^{-E[h(x)]/c(\psi)} \quad c(\psi) = 2H(\psi - 1) - \frac{2(\psi-1)}{\psi}$$

- Cortes al azar (variable y umbral) hasta dejar un punto solo: la **longitud del camino**  $h(x)$ .
- Muchos árboles, cada uno con una submuestra  $\psi$  (256 por defecto).
- $s$  cerca de 1: anomalía; alrededor de 0.5 o menos: normal.

## Ojo con el signo

En scikit-learn, `score_samples` devuelve  $-s$ : más bajo = más anómalo.

# 3 cortes para la anomalía, 14 para un punto normal



# Anómalo respecto de su vecindario

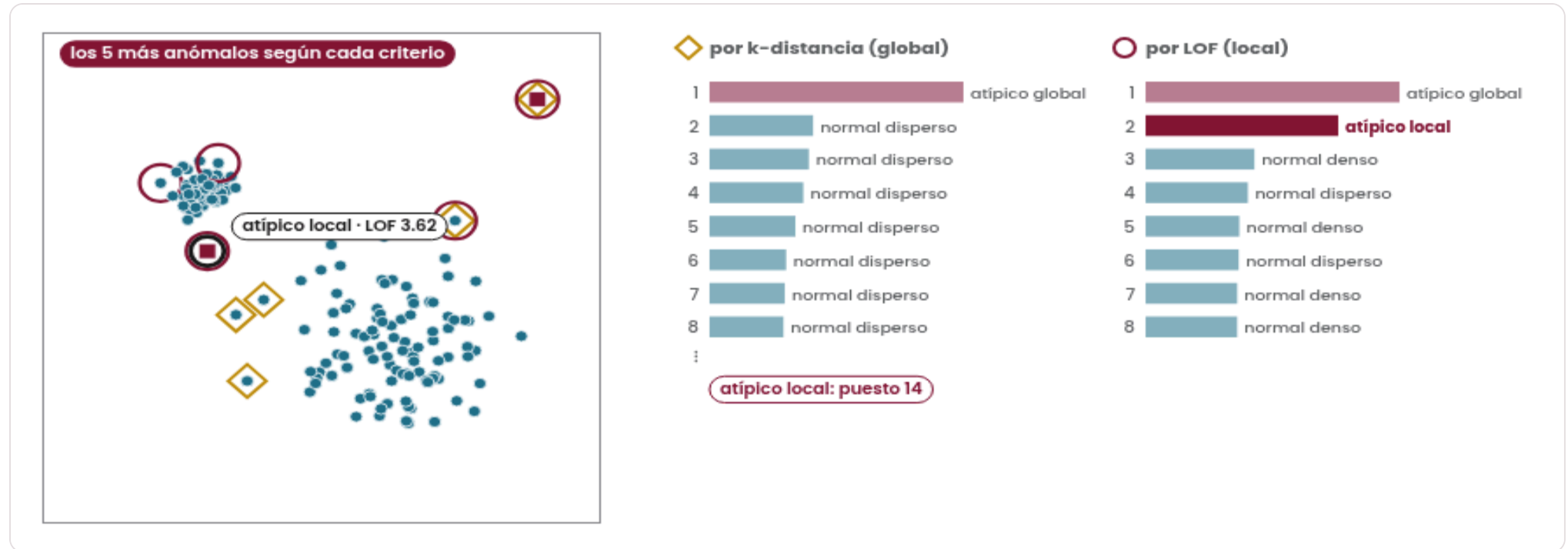
$$\text{lrd}(p) = \left( \frac{1}{k} \sum_{o \in N_k(p)} \max\{\text{k-dist}(o), d(p, o)\} \right)^{-1} \quad \text{LOF}(p) = \frac{\overline{\text{lrd}}(N_k(p))}{\text{lrd}(p)}$$

- Densidad local del punto frente a la de sus  $k$  vecinos.
- $\text{LOF} \approx 1$ : tan denso como su vecindario. Claramente  $> 1$ : atípico local.
- `n_neighbors` (20 por defecto) fija la escala; `novelty=True` para datos nuevos.

## Pregunta

Un punto normal de un grupo muy disperso tiene vecinos lejanos. ¿Su LOF es alto?

# Puesto 2 por LOF, puesto 14 por distancia



## Lo raro no siempre es lo malicioso

| PR-AUC (azar = 0.034) | Entrenamiento contaminado | Solo tráfico normal |
|-----------------------|---------------------------|---------------------|
| Máximo  z  (baseline) | 0.514                     | <b>0.906</b>        |
| Isolation Forest      | 0.441                     | 0.714               |
| LOF (k = 35)          | 0.075                     | 0.553               |

**smurf** (más del 70 % de los ataques): miles de conexiones casi idénticas forman una región **densa** y pasan por normales.

Para un recall del 80 %: 24 falsas alarmas por 10 000 normales con el baseline, **175** con Isolation Forest.

## El umbral lo fija el equipo que revisa

| Alertas por 10 000 conexiones | Recall       | Precisión |
|-------------------------------|--------------|-----------|
| 50                            | 0.140        | 0.940     |
| 200                           | 0.527        | 0.884     |
| 300                           | <b>0.823</b> | 0.921     |
| 400                           | 0.940        | 0.788     |
| 600                           | 0.985        | 0.551     |

≈ 340 ataques reales por lote: con menos capacidad, el recall está limitado.

Cada alerta cuesta horas de analista; el exceso produce **fatiga de alertas**.

"Con 300 alertas por lote detectamos el 82 %": así se conversa con el equipo.

BLOQUE 5 · 2:40–3:00

# Ética de segmentar y marcar personas

Sin etiquetas no significa sin decisiones.

## La edad no entró al modelo, pero está en los segmentos

| Segmento                        | Razón de<br>representación · 60+ |
|---------------------------------|----------------------------------|
| Con hipoteca, nunca contactados | 0.26                             |
| Sin hipoteca ni préstamo        | 1.74                             |
| Contactados antes               | 1.66                             |
| Con préstamo personal           | 0.34                             |

- Razón = proporción en el segmento / proporción en la población (1 = igual).
- Hipoteca, saldo y contactos previos se relacionan con la etapa de vida: son **proxies**.
- Las variables sensibles se **conservan para auditar**: sin ellas no veríamos el desbalance.
- Priorizar llamadas no es lo mismo que fijar tasas: el riesgo depende del **uso**.

# Ser "raro" no es ser riesgoso

- 1 **Impacto desigual:** clientes nuevos, otra región o turnos nocturnos son raros por definición y pueden recibir más alertas sin más riesgo.
- 2 **Mide por grupo:** tasa de alertas y de falsas alarmas, y su razón frente al grupo de referencia.
- 3 **Estigmatización:** nombres de segmentos y zonas marcadas pueden reproducir exclusiones (*redlining*).
- 4 **Diseño:** uso previsto documentado, revisión humana antes de actuar y posibilidad de apelar.

# Reducción de dimensionalidad, anomalías e impacto por subgrupos

- Datos: **default-of-credit-card-clients** (OpenML 42477): 30 000 clientes, 23 variables + incumplimiento.
- Escalamiento justificado; PCA con varianza y reconstrucción.
- t-SNE solo exploratorio ( $\geq 2$  perplejidades).
- K-means o DBSCAN validado y estable.
- Isolation Forest **y** LOF, comparados, con umbral justificado.
- Composición de clusters y anomalías por sexo, educación y edad, con tasas y razones.
- IA agéntica o de chat permitida, **declarada**.

## Rúbrica (100 pts)

Reproducibilidad 10 · PCA 15 · t-SNE 10 · Segmentación 15 · Anomalías IF/LOF y umbral 15 · **Subgrupos y riesgos éticos 25** · Comunicación 10

## Antes de irte, responde en el chat

- 1 Un informe dice que en el mapa de t-SNE el segmento A está "muy lejos" del B.  
¿Qué le responderías?
- 2 Tu Isolation Forest usa `contamination=0.05`. ¿Con qué criterio elegirías el umbral en su lugar?

A CONTINUACIÓN · LABORATORIO GUIADO DE 1.5 H

# Ahora, a segmentar clientes de verdad

E3.1 · Nivel 3: Bank Marketing y composición por subgrupo

Tarea 3 · vie 16/10